Tier 1 Full-Repository Engineering Task Guide

Generic Harbor Packaging, Prompt, Evaluation, Reward, and Training Design

1. What a Tier 1 Task Is

A Tier 1 repo-engineering task gives an agent one protected session in a complete production repository. The briefing resembles a real issue: it explains observable behavior, compatibility constraints, and required evidence, while leaving diagnosis and implementation open. The final repository is graded by executable deterministic checks. Optional judges explain quality, but they cannot convert broken behavior into a pass.

> The benchmark asks whether the final repository satisfies a real engineering contract, not whether the submitted diff matches a reference patch. Any implementation may pass when it satisfies the required behavior, regressions, integration contract, integrity boundary, and any architecture requirement clearly stated in the prompt.

Use the generic task identity <benchmark>/<task-slug> and replace it with the published package name. The task must point to a protected complete production repository at an exact pre-change commit; those placeholders describe the contract and are not agent-facing prompt text.

Property	Tier 1 requirement	Reason
Repository	Complete, production-scale worktree at an exact pre-change commit	Preserves navigation, build, test, and integration complexity
Session	One agent context without staged reviewer feedback	Measures full issue-to-patch execution in one run
Prompt	Natural behavior-first engineering brief	Avoids revealing a PR, patch recipe, or hidden-test map
Success	All critical deterministic gates pass	A judge cannot rescue incorrect behavior
Partial signal	Weighted, labeled training vector	Near misses remain useful without being called solved
Target headroom	Roughly 20-60% strict frontier solve rate after calibration	Avoids trivial or arbitrary tasks

2. End-to-End Flow

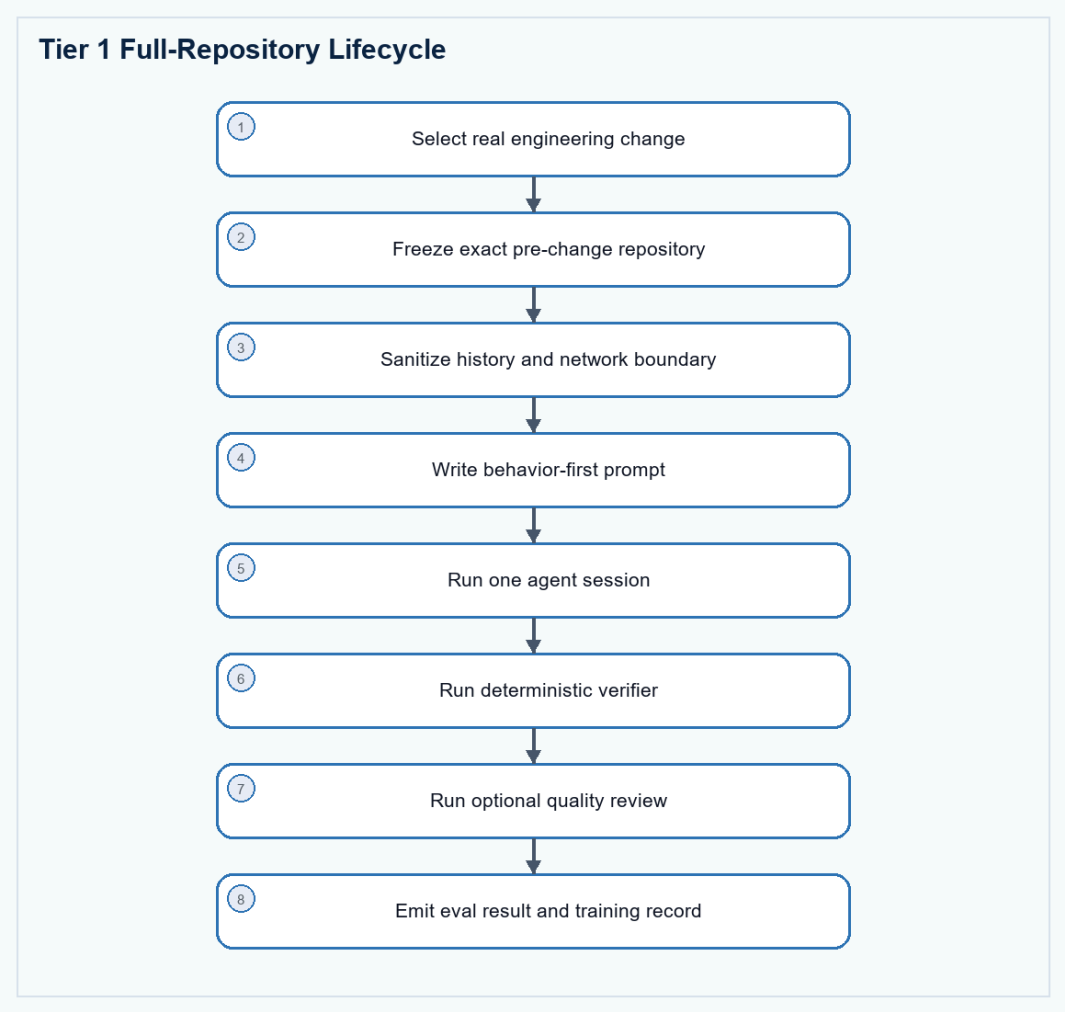

Figure 1. A real repository change becomes a protected single-session task and a labeled result.

The source change provides realism and an author Oracle, but it is not the verifier. Authors add independent hidden variants, regression coverage, integrity checks, and near-miss patches so public change recall alone cannot satisfy the benchmark.

3. Run Procedure

Run no-op, Oracle, and model trials as separate Harbor jobs. Keep unique job names and verify the actual agent and model in result.json before interpreting a score.

```
# Run from the directory that contains the task folder.
harbor run -p ./<task-directory> -a nop \
  --job-name <task>-nop

harbor run -p ./<task-directory> -a oracle \
  --job-name <task>-oracle
```

```
harbor run -p ./<task-directory> \
  -a terminus-2 -m <provider/model> \
  --job-name <task>-model

# Inspect the jobs in Harbor's local viewer.
harbor view ./jobs
```

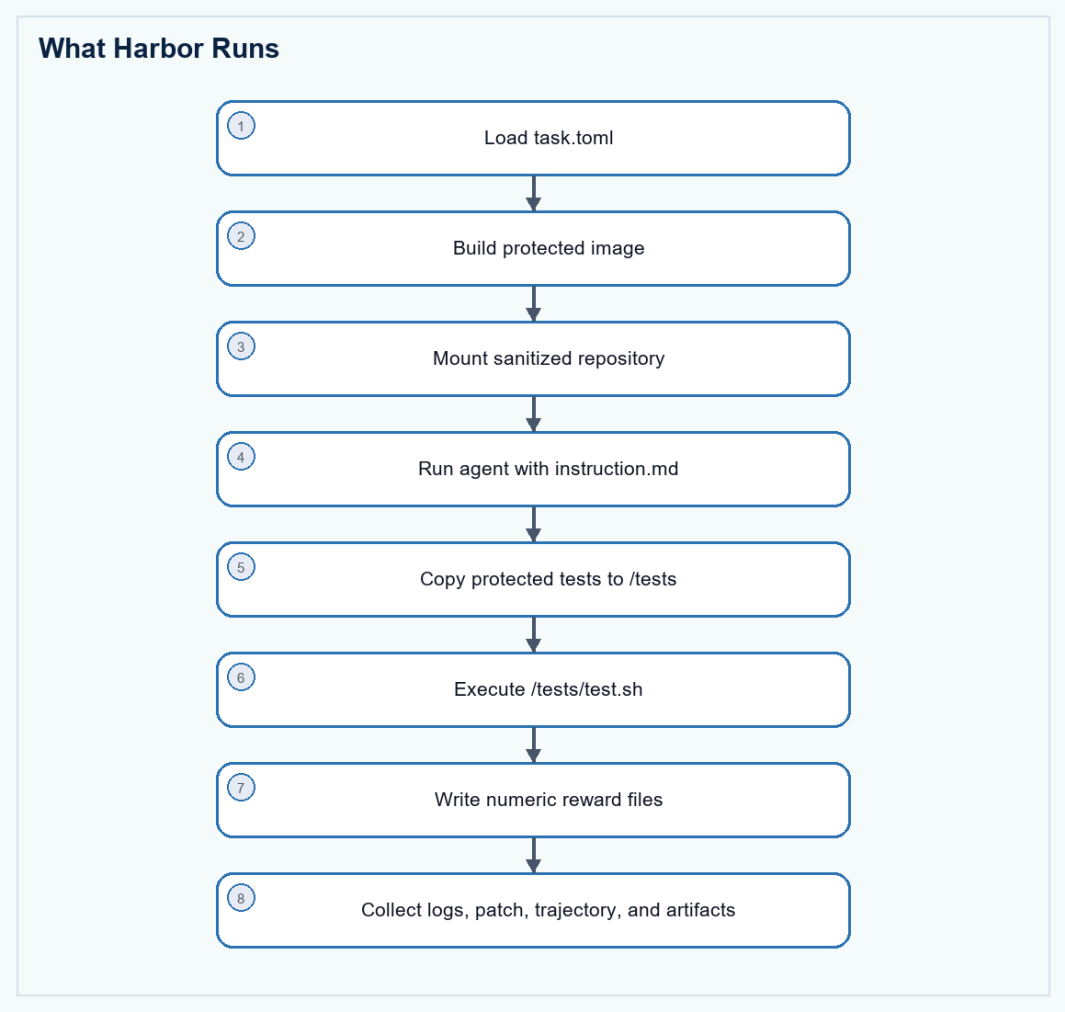

Figure 2. Harbor's task, agent, verifier, and artifact sequence.

Where results appear

```
/root/harbor-jobs/<job-name>/
`-- <task-name>__<trial-id>/
    |-- result.json
    |-- agent/trajectory.json
    |-- verifier/reward.json
    |-- verifier/reward-details.json
    |-- verifier/test_output.log
    +-- artifacts/
```

- Read result.json first for agent identity, model, completion state, and aggregate rewards.
- Read reward-details.json and test_output.log for the exact failed check family.

- Inspect trajectory.json and patch.diff before deciding whether a failure is a model miss, verifier defect, timeout, contamination, or infrastructure problem.

4. Recommended Task Package

Generic Tier 1 Harbor Package

```
<task-name>/
  |-- task.toml                        # Harbor schema, metadata, limits, network, artifacts
  |-- instruction.md                   # only the natural engineering brief shown to the agent
  |-- README.md                        # optional author-facing run and maintenance notes
  |-- environment/
  |   +-- Dockerfile                   # full repository, pinned toolchain, cached dependencies
  |-- solution/
  |   |-- solve.sh                     # author-only, repeatable Oracle entrypoint
  |   |-- golden.patch                 # optional author-only reference patch
  |   +-- provenance.json              # private source PR and exact commit provenance
  +-- tests/
      |-- test.sh                      # required Harbor verifier entrypoint
      |-- reward.toml                  # deterministic reward aggregation
      |-- sync_reward.py               # strict RewardKit output plus training-vector merge
      |-- test_reward_contract.py      # root-level discoverable packaging smoke tests
      |-- judge_prompt.md               # bounded common prompt for optional judges
      |-- common/
      |   |-- fixtures.py              # local fixtures and hidden variants
      |   +-- evidence.py              # safe result and trajectory staging helpers
      |-- deterministic/
      |   |-- engine.py                # executable build, behavior, regression and integrity prepass
      |   |-- check.py                 # RewardKit adapter over precomputed executable evidence
      |   |-- test_behavior.py         # fail-to-pass and hidden behavior cases
      |   |-- test_regression.py       # pass-to-pass and repository suites
      |   |-- test_integration.py      # cross-component, data, schema, migration checks
      |   +-- test_integrity.py        # contamination and verifier-boundary checks
      |-- process/
      |   |-- check.py                 # executable artifact and process gates
      |   +-- judge.toml               # optional trajectory-quality review
      |-- merge_readiness/
      |   +-- judge.toml               # optional maintainability and repository-taste review
      |-- validity/
      |   +-- judge.toml               # optional cheating and failure-class review
      |-- task_quality/
      |   +-- judge.toml               # author-facing prompt/verifier audit
      +-- near_miss/
          |-- README.md                # expected failure labels
          +-- *.patch                  # seeded partial and shortcut solutions
```

Figure 3. Recommended Tier 1 package with discoverable deterministic tests and separate optional judge dimensions.


```
<task-name>/
  |-- task.toml                        # Harbor schema, metadata, limits, network, artifacts
  |-- instruction.md                   # only the natural engineering brief shown to the agent
  |-- README.md                        # optional author-facing run and maintenance notes
  |-- environment/
  |   +-- Dockerfile                   # full repository, pinned toolchain, cached dependencies
  |-- solution/
  |   |-- solve.sh                     # author-only, repeatable Oracle entrypoint
  |   |-- golden.patch                 # optional author-only reference patch
  |   +-- provenance.json              # private source PR and exact commit provenance
  +-- tests/
      |-- test.sh                      # required Harbor verifier entrypoint
      |-- reward.toml                  # deterministic reward aggregation
      |-- sync_reward.py               # strict RewardKit output plus training-vector merge
      |-- test_reward_contract.py      # root-level discoverable packaging smoke tests
      |-- judge_prompt.md               # bounded common prompt for optional judges
      |-- common/
```

```
|   |-- fixtures.py               # local fixtures and hidden variants
|   +-- evidence.py               # safe result and trajectory staging helpers
|-- deterministic/
|   |-- engine.py                 # executable build, behavior, regression and integrity prepass
|   |-- check.py                  # RewardKit adapter over precomputed executable evidence
|   |-- test_behavior.py          # fail-to-pass and hidden behavior cases
|   |-- test_regression.py        # pass-to-pass and repository suites
|   |-- test_integration.py       # cross-component, data, schema, migration checks
|   +-- test_integrity.py         # contamination and verifier-boundary checks
|-- process/
|   |-- check.py                  # executable artifact and process gates
|   +-- judge.toml                # optional trajectory-quality review
|-- merge_readiness/
|   +-- judge.toml                # optional maintainability and repository-taste review
|-- validity/
|   +-- judge.toml                # optional cheating and failure-class review
|-- task_quality/
|   +-- judge.toml                # author-facing prompt/verifier audit
+-- near_miss/
    |-- README.md                 # expected failure labels
    +-- *.patch                   # seeded partial and shortcut solutions
```

Why this clears the warnings

Set metadata.benchmark_type to swe-bench for this repo-engineering family. Keep tests/test.sh, add tests/reward.toml, and place executable pytest cases in conventionally named tests/deterministic/test_*.py modules. Use one environment-level allow_internet = false baseline and omit phase-level network_mode overrides. This avoids asking providers to change network policy after the container has started.

File responsibilities

Path	Role	Must contain	Must not contain
task.toml	Harness contract	Schema, task identity, benchmark type, timeouts, resources, network and artifacts	Secrets, source PR URL, or unsupported fields
instruction.md	Agent brief	Symptom, invariant, preserved behavior, constraints and executable evidence	Golden function, exact file list, PR number, hidden test names or patch recipe
environment/Dockerfile	Reproducible runtime	Sanitized full repo, pinned toolchain, cached dependencies and output directories	Reachable future history, remotes, solution, tests or scored network installs
solution/	Author Oracle	Repeatable solve.sh, optional patch, private provenance	Anything mounted for the solving agent
tests/test.sh	Verifier entrypoint	Absolute paths, deterministic execution, numeric fail-safe reward and bounded logs	Network installs, swallowed missing tests or nonnumeric rewards
tests/deterministic/test_*.py	Executable truth	Hidden behavior, regressions, integration, robustness and integrity assertions	Golden-diff comparison or only the public example
judge.toml	Optional semantic review	Bounded files, explicit rubric and no-binary-rescue rule	Unbounded repository access or authority over deterministic truth

5. Harbor Metadata and Warning Fixes

Use Harbor's current nested task schema. metadata.benchmark_type is required by the benchmark validator used here even though Harbor allows arbitrary metadata. For a Tier 1 repository-engineering task, declare swe-bench unless the task genuinely belongs to another listed benchmark family.

```
schema_version = "1.3"


artifacts = [
  "/app/output/reproducer.sh",
  "/app/output/diagnosis.json",
  "/app/output/verification.json",
]


[task]
name = "<benchmark>/<task-slug>"
description = "A full-repository engineering task stated as an observable issue."
keywords = ["repo-engineering", "full-repository", "tier-1", "<language>"]


[metadata]
benchmark_type = "swe-bench"
category = "software-engineering"
language = "<language>"
tier = "core-repo-engineering"
repository = "<owner/repository>"
base_commit = "<exact-pre-change-commit>"
expert_time_estimate_hours = 12.0


[agent]
timeout_sec = 21600.0


[verifier]
timeout_sec = 5400.0


[environment]
build_timeout_sec = 7200.0
allow_internet = false
cpus = 8
memory_mb = 16384
storage_mb = 65536
```

Warning	Cause	Minimal correction	Verification
Missing [metadata].benchmark_type	The task has metadata but no recognized benchmark family	Add benchmark_type = "swe-bench" under [metadata]	Reload or lint the task and confirm the warning disappears
No tests found under tests/	Only a custom check.py is present, filenames are not discoverable, or tests were omitted from the archive	Keep tests/test.sh and add at least one real tests/deterministic/test_*.py module	List the packaged archive and run the no-op verifier
Agent network policy differs from baseline	agent or verifier network_mode overrides disagree with a fixed environment that cannot change policy after startup	Remove phase-level network_mode fields and set environment.allow_internet = false once	Inspect task.toml and confirm agent and verifier inherit the same offline baseline
Verifier produced no reward	test.sh exited before writing a reward	Always write numeric reward.json or reward.txt; fall back to zero on verifier error	Force one test failure and confirm Harbor records zero

Package pre-flight

```
# Required Harbor entrypoints and discoverable deterministic tests.
test -f task.toml
test -f instruction.md
test -f environment/Dockerfile
test -f solution/solve.sh
test -f tests/test.sh
test -f tests/reward.toml
find tests/deterministic -type f -name 'test_*.py' -print -quit | grep -q .
```

```
# Metadata warning must be absent for this task family.
grep -q '^benchmark_type = "swe-bench"$' task.toml

# Use one fixed offline environment baseline. Phase-level overrides can trigger
# a provider warning when network policy cannot change after container start.
grep -q '^allow_internet = false$' task.toml
! grep -q '^network_mode' task.toml

# Scripts should use Unix line endings and be executable in the packaged task.
file tests/test.sh solution/solve.sh
```

Archive note

Before publishing a ZIP or registry package, inspect the archive itself. Confirm tests/test.sh, reward.toml, and test_*.py files are present, shell scripts use LF line endings, and solution/verifier files are not copied into the agent image.

6. Environment and Dependencies

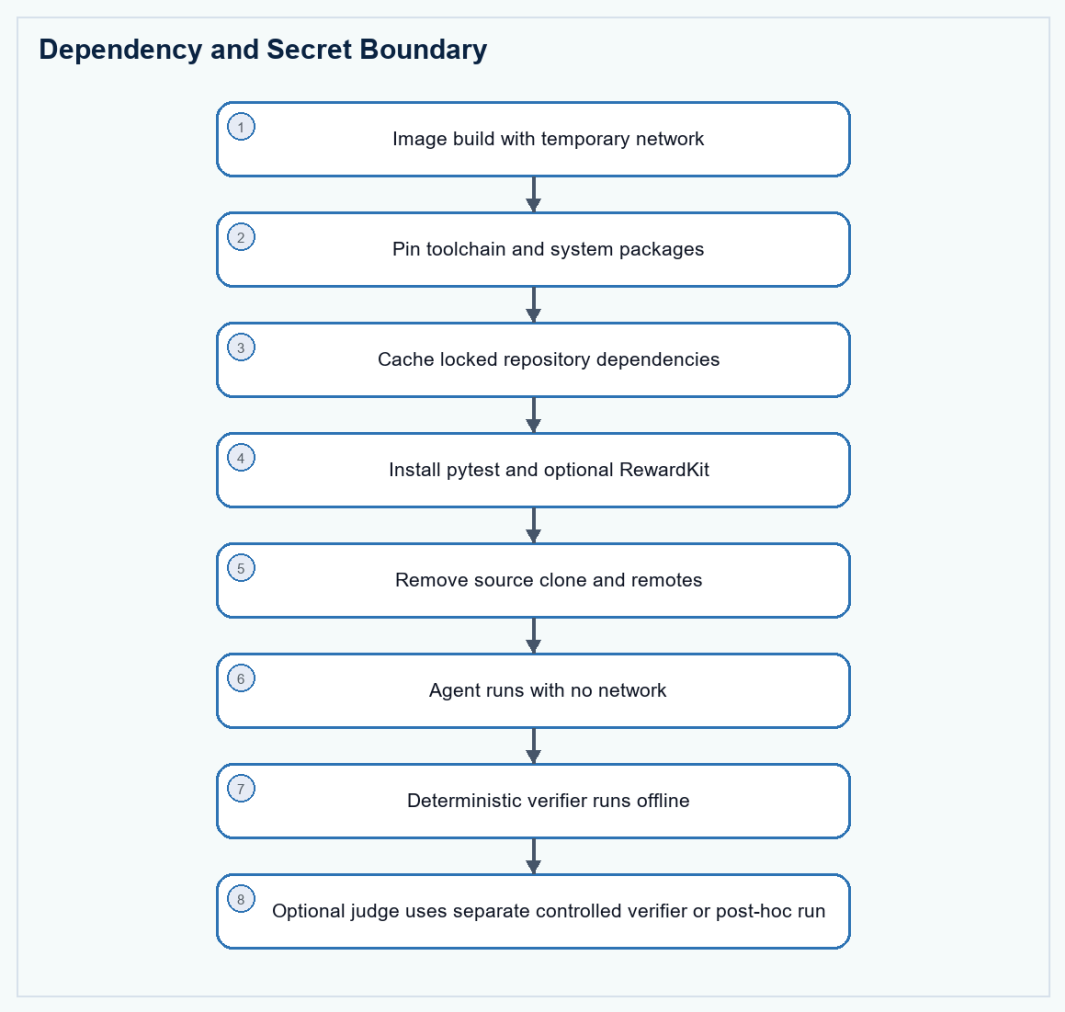

Figure 4. Dependencies are installed before scoring; judge credentials remain outside the agent boundary.

Dependency	Location	Pin or requirement	Purpose
Repository toolchain	Agent/environment image	Pinned compiler, package manager, native libraries and build tools	Build and test the real repository
Locked repository dependencies	Environment image cache	Language lockfile dependencies and required runtime assets	Keep every scored run offline and repeatable
Python	Isolated verifier runtime	Python 3.12 or newer for harbor-rewardkit 0.1.7	Keep verifier packages compatible with non-Python repository images
pytest	Verifier runtime	Pin a validated version, for example 8.3.5	Discover and execute test_*.py behavior and regression suites
harbor-rewardkit	Verifier runtime	Pin the tested package; use [all] only when judge/document extras are needed	Weighted criteria, multiple reward dimensions and optional judges
Git/ripgrep/jq	Environment image	Pinned OS packages where practical	Repository inspection, integrity checks and JSON handling
Judge provider client/key	Separate or post-hoc verifier only	Runtime secret plus controlled network access	Optional process, validity and merge-readiness review

Generic Dockerfile pattern

```
FROM <pinned-language-image>


ARG BASE_COMMIT=<exact-pre-change-commit>
ARG PYTHON_VERSION=3.12
ARG PYTEST_VERSION=8.3.5
ARG REWARDKIT_VERSION=0.1.7


# Install the repository toolchain during image build. RewardKit 0.1.7 needs
# Python 3.12 or newer, so do not assume the language base image is sufficient.
RUN <system-package-install> git ca-certificates ripgrep jq tmux
RUN <install-or-bootstrap-python-3.12>
RUN python3.12 -m venv /opt/verifier-venv \
 && /opt/verifier-venv/bin/python -m pip install --no-cache-dir \
    "pytest==${PYTEST_VERSION}" \
    "harbor-rewardkit[all]==${REWARDKIT_VERSION}" \
 && /opt/verifier-venv/bin/python -c "import pytest, rewardkit" \
 && /opt/verifier-venv/bin/rewardkit --help >/dev/null


ENV PATH=/opt/verifier-venv/bin:<repository-toolchain-path>:/usr/local/bin:/usr/bin:/bin


# Fetch only while building, then export a sanitized worktree without remotes,
# future commits, PR identifiers, solution files, or verifier files.
RUN git clone <repository-url> /tmp/source \
 && git -C /tmp/source checkout --detach "${BASE_COMMIT}" \
 && mkdir -p /testbed \
 && git -C /tmp/source archive "${BASE_COMMIT}" | tar -x -C /testbed \
 && rm -rf /tmp/source


WORKDIR /testbed
RUN <cache-locked-repository-dependencies-and-build-targets>
RUN git init && git add -A \
 && git -c user.name=benchmark -c user.email=benchmark@invalid \
    commit -m "sanitized benchmark base"


RUN mkdir -p /app/output /logs/agent /logs/verifier
CMD ["sleep", "infinity"]
```

- RewardKit is optional for a simple single numeric reward, but recommended when the task needs weighted criteria, multiple training dimensions, or LLM judges.
- Check the interpreter requirement before installing RewardKit. The validated harbor-rewardkit 0.1.7 setup requires Python 3.12 or newer; a Rust, C++, or older Linux base image may ship only Python 3.11.
- Create an isolated verifier virtual environment and put it first on PATH so pytest and rewardkit use the intended interpreter without changing the repository toolchain.
- Install verifier dependencies during image build or in a separate verifier image. Never use pip, cargo, npm, Maven, or apt network installs during scored verification.
- If judges require an API key, inject it only into a separate or post-hoc verifier and allow only the required provider host. The solving agent must not inherit the key or judge network.
- Pin versions that have passed Oracle and no-op validation. The example versions are a starting point, not an instruction to upgrade an already stable task.

7. Prompt Contract

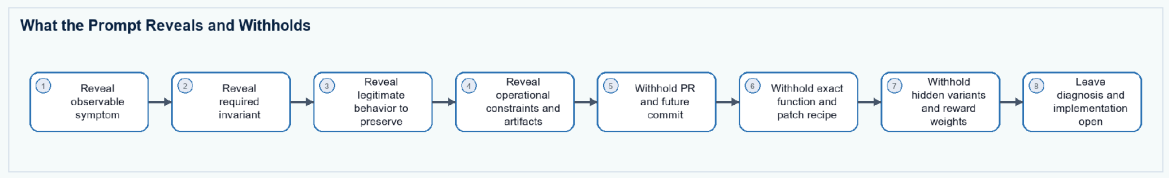

Figure 5. A realistic brief gives sufficient behavioral scope without disclosing the solution.

Generic instruction template

```
# <Natural issue title>


The complete repository is available at `/testbed`.


Describe the observed user or system problem in normal issue language. Include
one small example only when it helps reproduce the symptom. Do not name the
source pull request, golden commit, hidden test, exact function, or patch recipe.


## Required behavior


- State the invariant that must hold after the change.
- State legitimate behavior that must remain supported.
- State important cross-component, compatibility, data, migration, fault,
  performance, ordering, or concurrency constraints.
- Require existing unrelated behavior to remain unchanged.
- Require a coherent repository-level fix, not a hard-coded example.


## Executable engineering evidence


Create under `/app/output`:


1. `reproducer.sh`: an executable offline reproducer whose exit status reflects
   the required behavior.
2. `diagnosis.json`: subsystem, observed invariant, root cause, rejected
   alternatives, and files inspected.
3. `verification.json`: commands run, cases covered, regression suites, and
   remaining risk.


Do not access external services, task-author files, verifier files, Git remotes,
future history, or an upstream solution. Keep the patch mergeable in the full
repository.
```

Prompt acceptance checks

- A repository maintainer could plausibly file the brief without mentioning benchmarks or evaluation signals.
- The prompt clearly distinguishes behavior that must change from behavior that must remain supported.
- The prompt requires a repository-level outcome but does not list every file or implementation symbol.
- Every critical deterministic gate traces to a stated requirement or ordinary regression expectation.
- Required reports are secondary evidence; prose alone cannot rescue executable failure.

8. Deterministic Tests and Optional Judges

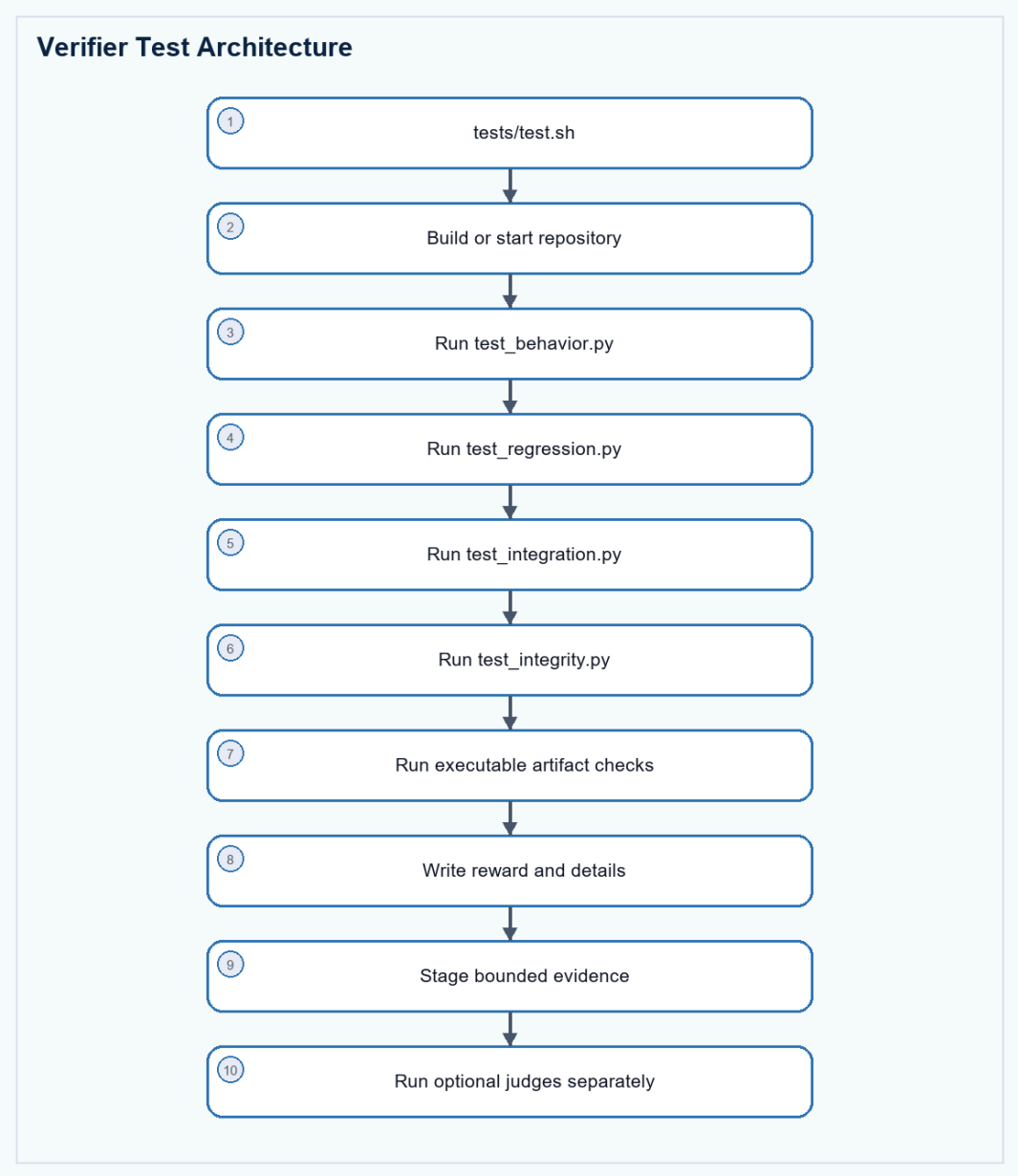

Figure 6. Deterministic truth is executed first; bounded semantic review remains separate.

Check family	Class	Recommended location	What passing proves
Build and startup	Deterministic	tests/deterministic/test_regression.py	The repository and target service or binary build offline
Fail-to-pass behavior	Deterministic	tests/deterministic/test_behavior.py	The original defect is fixed across hidden variants
Pass-to-pass safety	Deterministic	tests/deterministic/test_regression.py	Existing relevant tests and unrelated contracts still pass
Integration/data contract	Deterministic	tests/deterministic/test_integration.py	Callers, schemas, migrations, persistence, or protocols agree
Robustness/performance	Deterministic when stable	tests/deterministic/test_integration.py	Fault, concurrency, resource, rollback, or budget behavior holds

Check family	Class	Recommended location	What passing proves
Architecture requirement	Deterministic only when prompted	tests/deterministic/test_integration.py	A required obsolete path or dual model is actually removed
Executable evidence	Deterministic	tests/process/check.py	Reproducer and reports correspond to commands and outputs
Repository integrity	Deterministic	tests/deterministic/test_integrity.py	No solution, verifier, remote, history, or reward tampering occurred
Process quality	Non-deterministic	tests/process/judge.toml	The trajectory diagnosed, implemented, and verified coherently
Merge readiness	Non-deterministic	tests/merge_readiness/judge.toml	The strict-passing patch is idiomatic, focused, and reviewable
Trial validity	Non-deterministic	tests/validity/judge.toml	A pass or failure is genuine rather than leakage or infrastructure error
Task quality	Non-deterministic; author-facing	tests/task_quality/judge.toml	Prompt, hidden checks, artifacts, and intended skill agree

Behavior versus patch shape

Do not compare the candidate diff with golden.patch. Prefer executable behavior, repository tests, properties, metamorphic variants, fault injection, and integration checks. Use source-shape checks only when the prompt explicitly requires an architectural removal, public API, migration boundary, or security invariant that behavior alone cannot distinguish.

Deterministic test design

- Fail-to-pass: reproduce the original defect plus hidden variants with different names, ordering, values, state, topology, or timing.
- Pass-to-pass: run focused existing suites and unrelated contracts most likely to regress.
- Property or metamorphic: change irrelevant inputs, execution order, equivalent constraints, serialization form, or retry sequence while preserving the expected result.
- Integration: exercise real callers, persistence, schema, migration, protocol, CLI, service, or generated artifacts instead of isolated helper functions only.
- Performance and fault checks: use deterministic budgets, controlled clocks, seeded schedules, fixed fault points, and repeated Oracle baselines; exclude flaky checks from strict truth.
- Integrity: reject verifier edits, solution access, public remotes, future history, test replacement, reward tampering, and suspicious hard-coded hidden fixtures.

9. Reward Shape

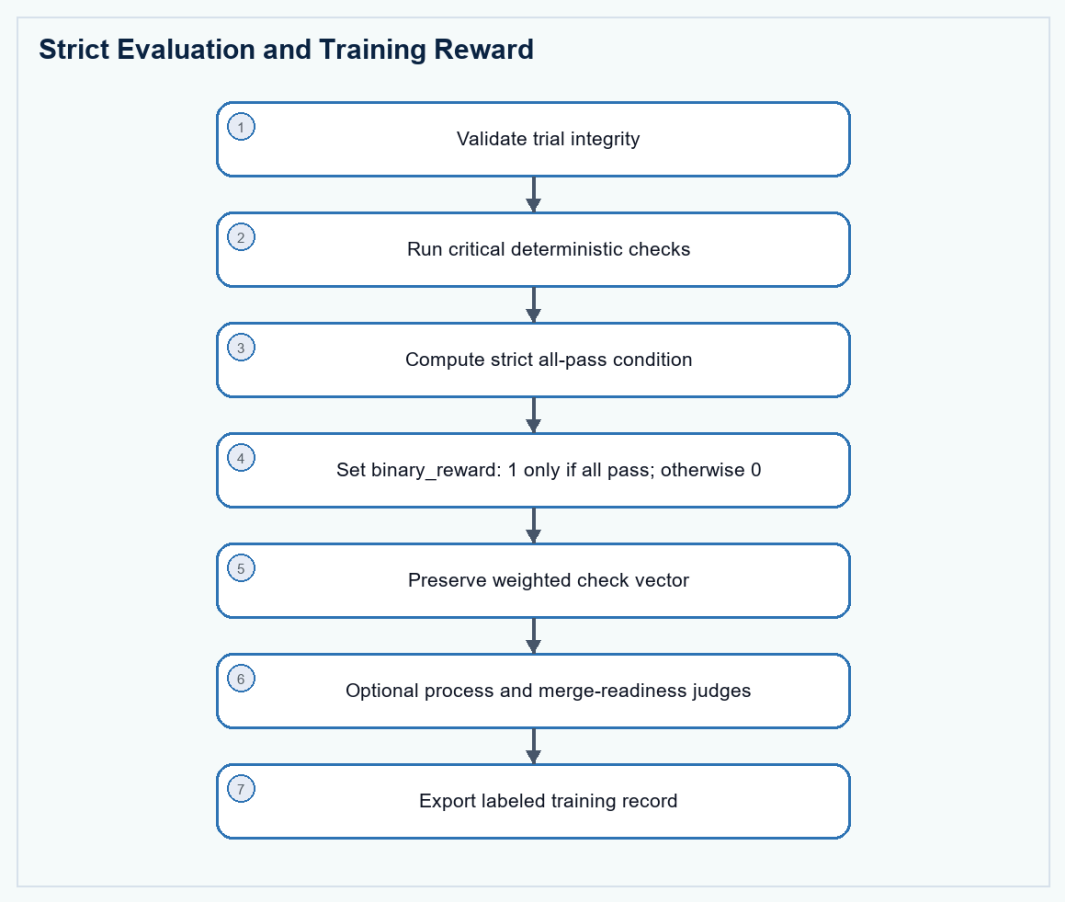

Figure 7. Strict success remains binary while valid partial work receives labeled training signals.

Output	Meaning	Use	Rule
binary_reward	Strict benchmark success	Leaderboard and eval pass rate	1 only when every critical deterministic gate passes
training_score	Weighted deterministic progress	Curriculum, credit assignment and near-miss ranking	Never reported as a solved task
valid_trial	Evaluation boundary remained intact	Training eligibility and quarantine	Zero for contamination or invalid harness state
fail_to_pass	Original and hidden behavior repaired	Behavioral learning signal	Derived only from executable checks
pass_to_pass	Relevant prior behavior preserved	Regression-safety signal	Separate from new behavior
repo_integration	Cross-component contract is coherent	Integration and migration training	Includes real callers or repository scenarios
merge_ready_score	Quality of a strict-passing patch	Preference data and review research	Zero or unreported unless deterministic success holds

Non-deterministic checks

Judges are useful for diagnosis quality, repository conventions, patch minimality, architecture taste, reviewability, and suspicious trajectory review. They should receive only bounded evidence: instruction, changed files, patch, selected logs, reward details, declared artifacts, and trajectory. Run repeated judges or human audits when a score matters. Judge disagreement is uncertainty, not deterministic truth.

> **Two separate reward passes**
>
> Run executable checks and the deterministic RewardKit suite inside the offline task. Run process, validity, task-quality, and merge-readiness judges in a separate controlled or post-hoc pass. Preserve reward and binary_reward from deterministic evaluation even when a judge is unavailable or disagrees.

Reward implementation templates

RewardKit aggregation

```
# Run this aggregation only over deterministic dimensions.
[[reward]]
name = "reward"
aggregation = "threshold"
threshold = 1.0


[[reward]]
name = "binary_reward"
aggregation = "threshold"
threshold = 1.0


[[reward]]
name = "training_score"
aggregation = "weighted_mean"
```

Verifier entrypoint pattern

```
#!/usr/bin/env bash
set -uo pipefail


RESULT_DIR="${RESULT_DIR:-/logs/verifier}"
TESTS_DIR="$(CDPATH= cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd)"
ENGINE="$TESTS_DIR/deterministic/engine.py"
SUITE_DIR="$RESULT_DIR/deterministic_suite"
REWARDKIT_OUT="$RESULT_DIR/rewardkit_deterministic"
mkdir -p "$RESULT_DIR" /app/output


# First run the real repository checks. The engine writes granular executable
# evidence; it must not fetch dependencies during a scored trial.
python3 "$ENGINE" >"$RESULT_DIR/engine_output.log" 2>&1 || true
cp "$RESULT_DIR/reward.json" "$RESULT_DIR/deterministic-engine-reward.json" 2>/dev/null || true
cp "$RESULT_DIR/reward-details.json" "$RESULT_DIR/deterministic-engine-details.json" 2>/dev/null || true


# Keep conventionally named tests visible to validators and human reviewers.
DETERMINISTIC_DETAILS_PATH="$RESULT_DIR/deterministic-engine-details.json" \
  PYTHONPATH="$TESTS_DIR/deterministic${PYTHONPATH:+:$PYTHONPATH}" \
  pytest -q -p no:cacheprovider "$TESTS_DIR/test_reward_contract.py" \
    "$TESTS_DIR/deterministic"/test_*.py \
    >"$RESULT_DIR/test_output.log" 2>&1 || true


# RewardKit aggregates only deterministic criteria here. Optional judges are
# deliberately excluded from this suite.
rm -rf "$SUITE_DIR" "$REWARDKIT_OUT"
mkdir -p "$SUITE_DIR/deterministic" "$REWARDKIT_OUT"
cp "$TESTS_DIR/reward.toml" "$SUITE_DIR/reward.toml"
cp "$TESTS_DIR/deterministic/check.py" "$SUITE_DIR/deterministic/check.py"
```

```
status=0
DETERMINISTIC_DETAILS_PATH="$RESULT_DIR/deterministic-engine-details.json" \
  rewardkit "$SUITE_DIR" --workspace /testbed \
  --output "$REWARDKIT_OUT/reward.json" \
  >>"$RESULT_DIR/test_output.log" 2>&1 || status=$?


# Merge strict RewardKit truth with the executable training vector. The sync
# script writes numeric zero on infrastructure failure instead of no reward.
python3 "$TESTS_DIR/sync_reward.py" "$RESULT_DIR" "$REWARDKIT_OUT" "$status"


# Optional process, validity and merge-readiness judges run separately or
# post-hoc. They must never overwrite binary_reward or rescue deterministic
# failure.
exit 0
```

10. Authoring and Acceptance

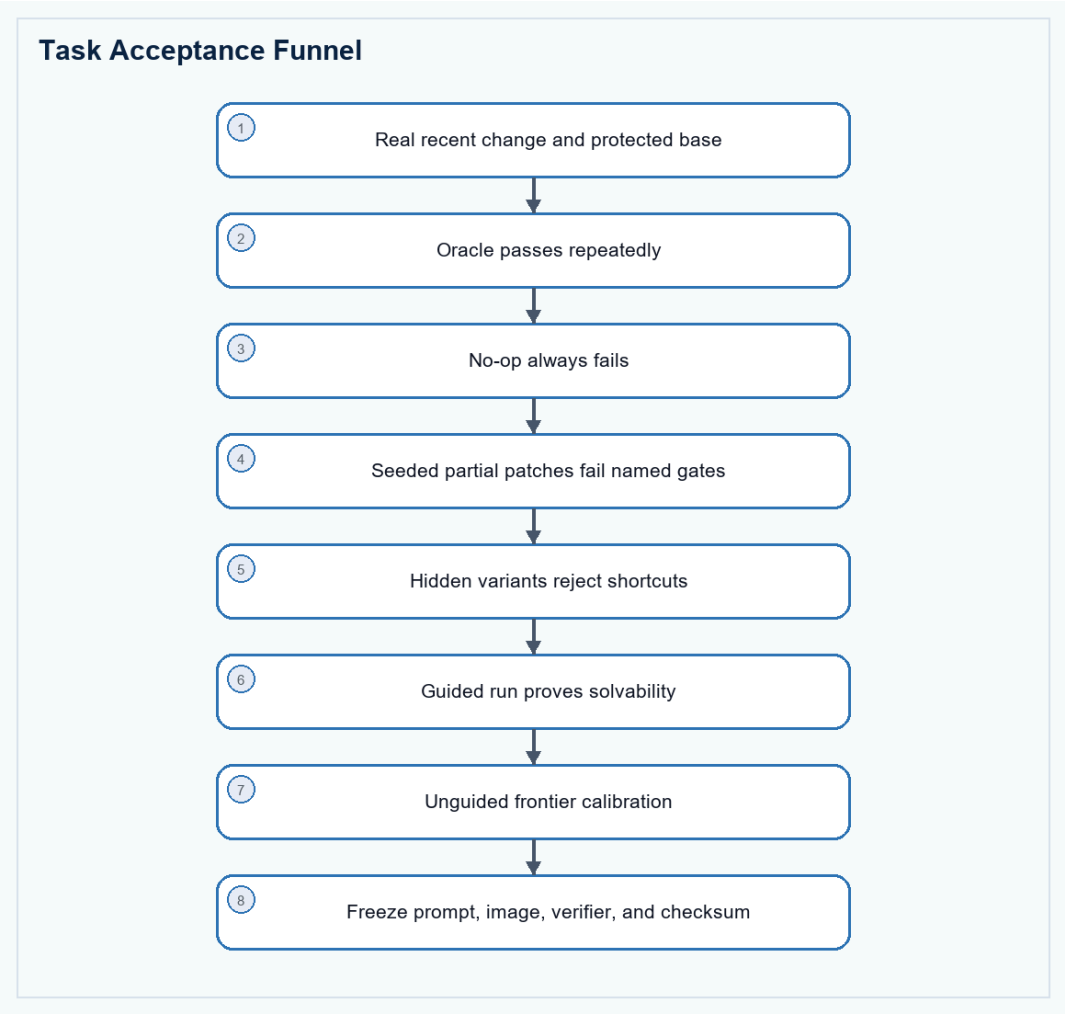

Figure 8. A task is accepted only after solvability, shortcut rejection, and frontier headroom are demonstrated.

- Select stateful, cross-component work involving runtime behavior, data contracts, migrations, concurrency, performance, faults, or compatibility rather than patch size alone.

- Keep the exact PR, issue, merge commit, and Oracle privately for provenance; remove all of them from the agent-visible image and prompt.
- Require Oracle success in repeated clean builds and no-op failure in every run.
- Seed at least three plausible near misses: narrow symptom patch, over-broad behavior change, and incomplete integration or regression update.
- Confirm hidden verifiers reject at least 90% of authored near-miss mutations before accepting the task.
- Calibrate multiple frontier agents and investigate trajectories, not only aggregate scores.
- Freeze the prompt, base image, verifier, dependency versions, and task checksum together.

11. Evaluation-to-Training Use

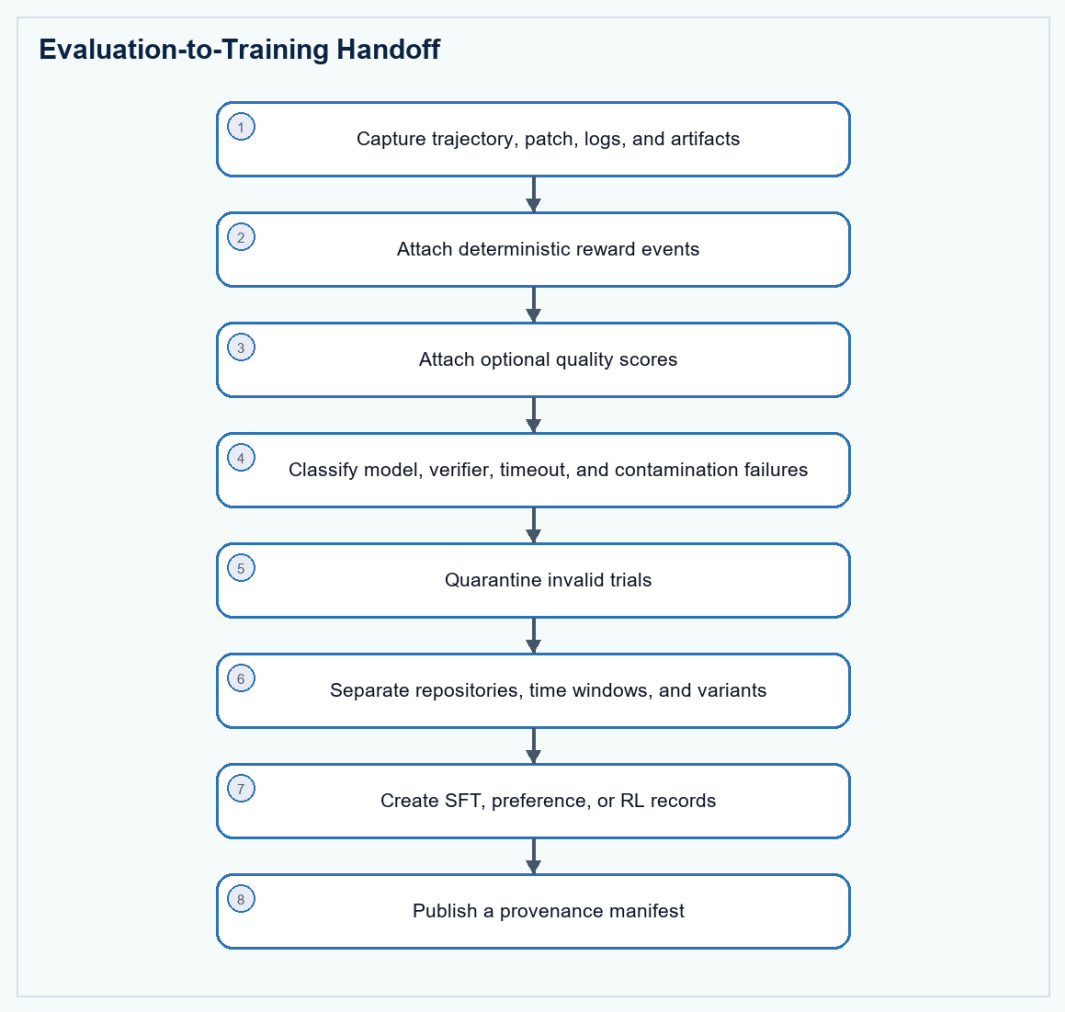

Figure 9. Valid trials become labeled training records without leaking evaluation instances.

- Export full trajectories, repository snapshots or patches, commands, test outputs, reward events, artifacts, failure labels, contamination labels, and optional judge reasoning.
- Retain valid failed trials and near misses because they show localization, implementation, regression, or integration boundaries.
- Create pairwise preferences only from comparable valid trials, such as strict pass versus near miss or focused patch versus bloated strict pass.

- Never train on an active evaluation instance. Separate training and evaluation by repository, time window, hidden variant, and task checksum.

12. Worked Example Retained from the Earlier Guide

Example, not template

The earlier UV task remains useful as a calibration case. It is based on a real Rust resolver migration, but the generic package and verifier rules in this guide should be applied to any suitable repository and language.

Example fact	Observed value	Lesson for generic Tier 1 design
Source	UV PR #20302 at its exact pre-change commit	Use real provenance privately and sanitize it from the solving environment
Scope	Resolver behavior, repository scenarios, CLI/settings, generated schema and documentation	Grade one contract across multiple repository representations
Oracle	Strict deterministic pass	The author solution must prove repeatable solvability
Frontier trial	13 of 16 checks; strict 0; weighted training score 0.70	Partial vectors explain progress without inflating pass rate
Failure families	Architecture cleanup, repository scenarios, documentation/schema consistency	A plausible local fix is not enough when the full repository remains inconsistent
Patch identity	No comparison with golden.patch	Evaluate behavior and declared architecture, not line-for-line similarity

Fresh-container validation of the package pattern

The finalized UV package was rebuilt and its verifier was run directly in fresh Docker containers. These values validate the packaging and reward path; they are an example calibration record, not universal reward targets for every Tier 1 task.

Trial	Observed result	What it proves
No-op	binary_reward 0.0; training_score 0.51; valid_trial 1.0; rewardkit_available 1.0	An untouched repository fails strict behavior while still producing a valid, useful partial training vector
Oracle	All 16 deterministic criteria passed; binary_reward, training_score, integration, regression and process values were 1.0	The author solution is solvable through the complete executable verifier and RewardKit path
Runtime	Python 3.12.13; pytest 8.3.5; harbor-rewardkit 0.1.7; Rust 1.97	A non-Python repository can carry a separate pinned verifier runtime without changing its main toolchain
Network policy	One environment allow_internet = false baseline; no agent or verifier network_mode override	Agent and verifier inherit one startup policy, avoiding unsupported phase policy changes
Discovery	Five conventionally named test_*.py modules plus tests/test.sh and reward.toml	The package is visible to Harbor-style validators and still retains a richer executable engine

13. Final Author Checklist

- task.toml uses schema_version 1.3 and declares metadata.benchmark_type = swe-bench.
- environment.allow_internet is false and agent/verifier sections do not override network policy.

- tests/test.sh, tests/reward.toml, and at least one tests/deterministic/test_*.py file are present in the packaged artifact.
- The verifier interpreter satisfies RewardKit's requirement and pytest/rewardkit resolve from the pinned isolated environment.
- The full repository and all locked dependencies build without network access during the scored run.
- The prompt states behavior, preserved contracts, constraints, and executable evidence without revealing provenance or a patch recipe.
- Critical deterministic checks cover fail-to-pass, pass-to-pass, integration, artifacts, and integrity.
- Source-shape checks exist only for explicitly required architecture or security boundaries.
- Optional judges are bounded, reproducible where possible, and unable to change binary truth.
- Oracle, no-op, seeded near misses, guided runs, and multiple frontier trials have all been reviewed.
- Every verifier exit path writes a numeric reward and enough logs to classify failure.
- Training exports are separated from active evaluation repositories, windows, variants, and checksums.

14. Source Map

Harbor task and RewardKit guidance was checked against the current official documentation on July 13, 2026. Source-change identity belongs in private author provenance or a worked example, not in the protected agent instruction.

Source	URL	Used for	Planning decision
SWE-bench	swebench.com	Issue-to-patch repository evaluation and regression tests	Keep executable behavior and pass-to-pass safety
SWE-Bench Pro	labs.scale.com	Professional repositories and reproducible task containers	Use real full repositories, strong tests and protected splits
Senior SWE-Bench	senior-swe-bench.snorkel.ai	Natural briefs, runtime verification, taste and difficult investigations	Keep deterministic correctness separate from merge readiness
Senior SWE-Bench tasks	github.com	Concrete Harbor-compatible task packages	Reuse package discipline while strengthening long-horizon and contamination controls
Harbor task documentation	harborframework.com	Task, agent, environment, verifier, reward and artifact model	Use one reproducible package and explicit reward files
Harbor task structure	harborframework.com	Required instruction, environment, solution, tests/test.sh and numeric reward contract	Keep test.sh as the stable verifier entrypoint and use absolute container paths
Harbor RewardKit	harborframework.com	Programmatic criteria, judge criteria, reward dimensions and reward.toml aggregation	Separate deterministic truth from optional quality review
Harbor task tutorial	harborframework.com	Discoverable test_*.py convention and Oracle/no-op workflow	Ship at least one conventionally named deterministic test module
UV worked example	github.com	Recent full-repository Rust resolver change used to calibrate the earlier guide	Retain as an example, not as the generic task template